



Day 1 — Jenkins & CI/CD Fundamentals

Building the foundation for automated software delivery.



Learning Objectives

1

CI/CD Concepts

Explain CI, Continuous Delivery, and Continuous Deployment

2

Jenkins Role

Understand where Jenkins fits into CI/CD workflows

3

Architecture

Describe Jenkins Controller–Agent architecture

4

Pipelines

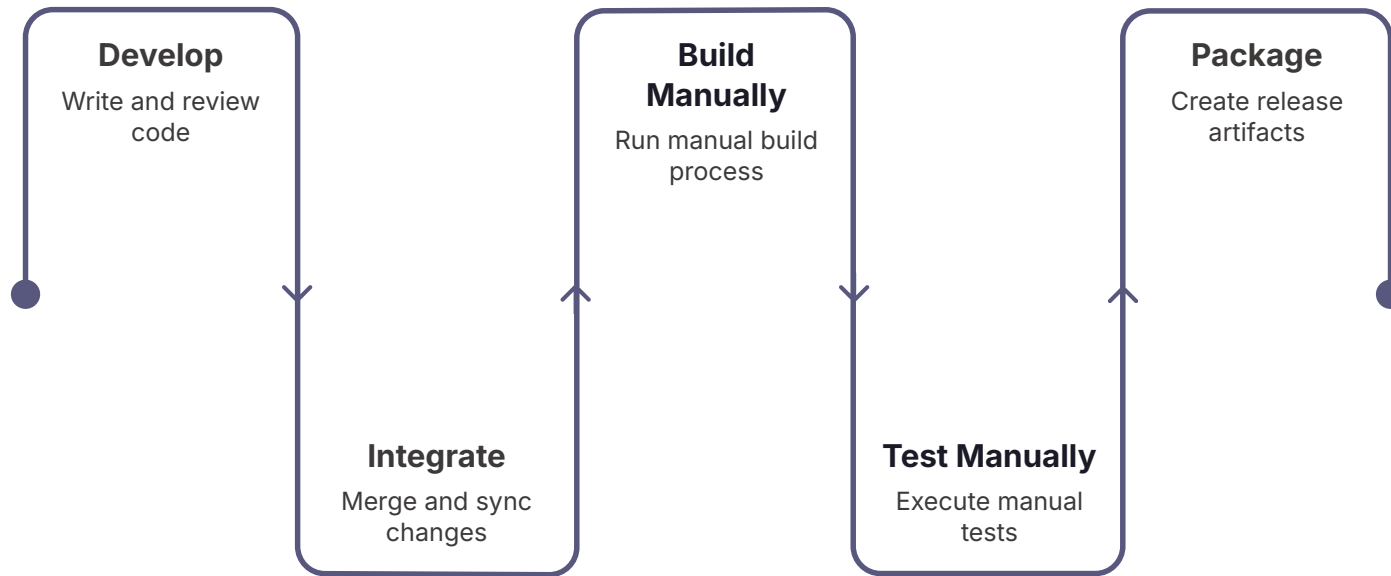
Compare Freestyle and Pipeline jobs; explain Pipeline-as-Code



Hands-on: Jenkins Job + Basic Pipeline

The Software Delivery Challenge

Traditional Delivery



Common Challenges

The Need: Automate repetitive software delivery activities and provide faster feedback.

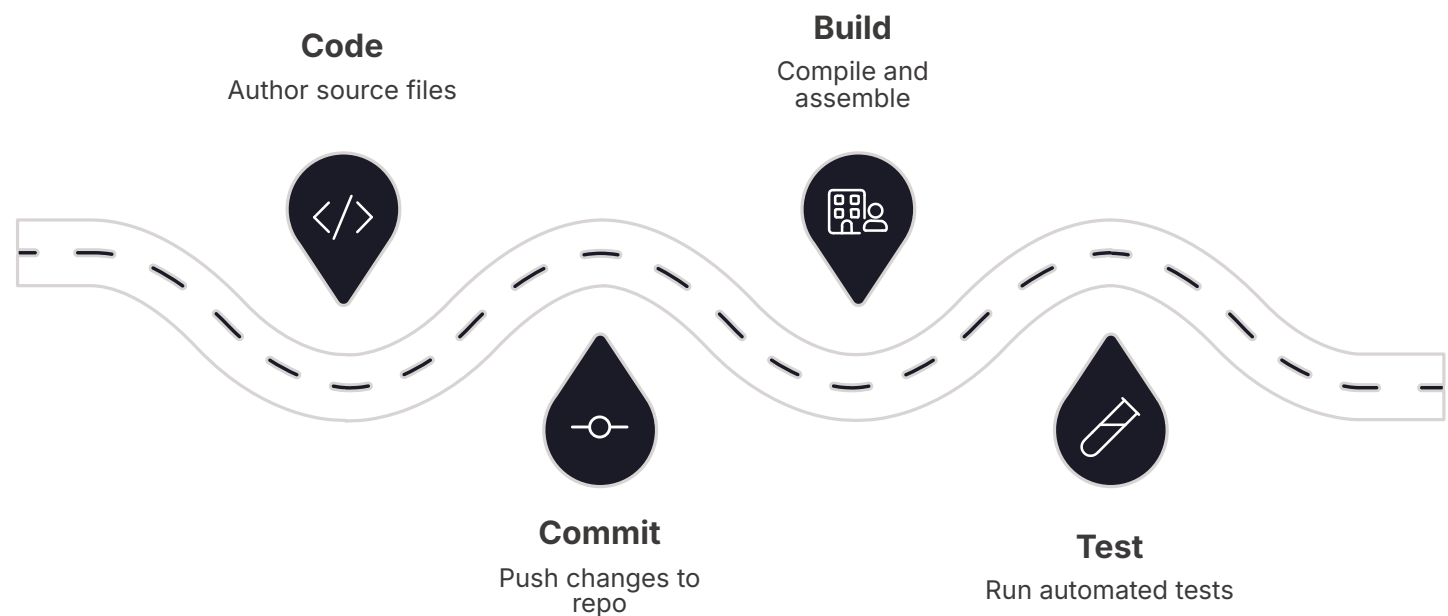
→ **Late integration issues & slow feedback**

→ **Manual, inconsistent builds & deployment errors**

→ **Environment differences & risky releases**

Understanding CI/CD

CI/CD automates the software delivery lifecycle — from code commit to deployment.



Automate Tasks

Eliminate repetitive manual steps

Early Detection

Detect problems before they grow

Faster Feedback

Deliver results to developers quickly

Reduce Risk

Repeatable processes, fewer release errors

Continuous Integration (CI)

The practice of **frequently integrating code changes** into a shared repository and automatically validating those changes.



Git Commit / Push



Automated Build



Automated Tests



Feedback



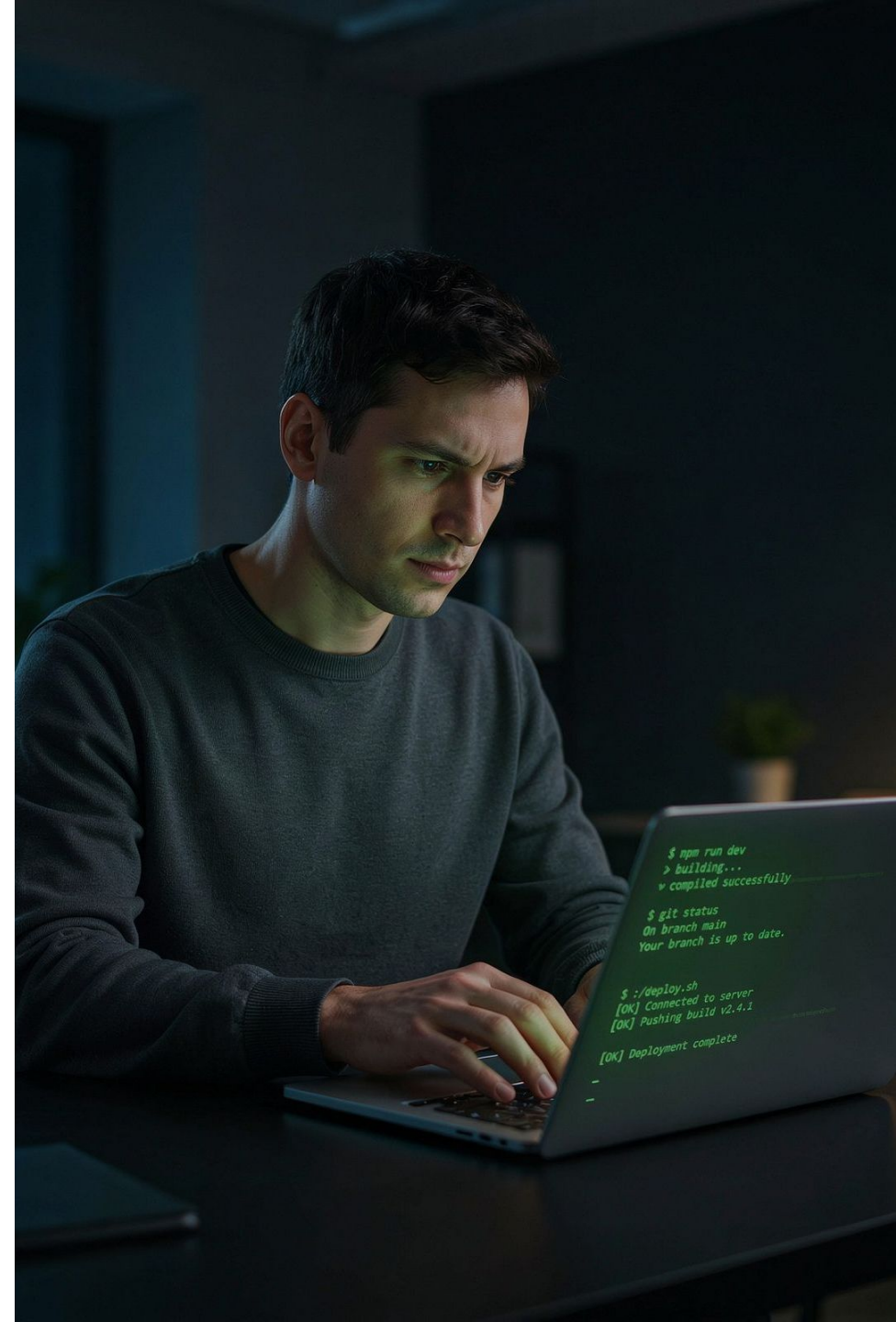
Early Defect Detection



Faster Feedback



Better Collaboration



Continuous Delivery vs. Continuous Deployment

CONTINUOUS DELIVERY

Always Release-Ready

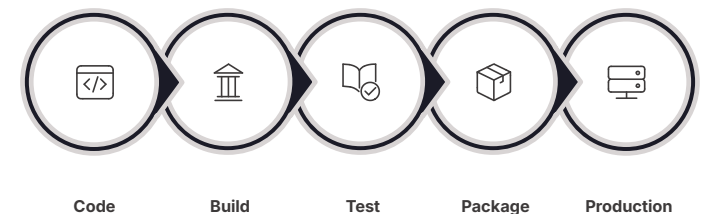
Software is always kept **ready for deployment**. A manual approval gate controls when it reaches production.



CONTINUOUS DEPLOYMENT

Fully Automated

Every change that passes automated checks is **automatically deployed** to production — no manual step required.



CI vs. Delivery vs. Deployment

Practice	Main Objective	Production Deployment
Continuous Integration	Integrate & validate code	Not necessarily
Continuous Delivery	Keep software release-ready	Manual decision
Continuous Deployment	Automatically release validated changes	Automatic

CI

Is the integrated code working?

Delivery

Is the application ready to release?

Deployment

Can validated changes automatically reach production?

What is Jenkins?



An Automation Server

Jenkins is an **automation server** commonly used to implement CI/CD workflows. It coordinates the entire delivery process.

Typical Integrations

Git / GitHub · Maven / Gradle ·
SonarQube · Artifactory · Docker ·
Deployment platforms

Build

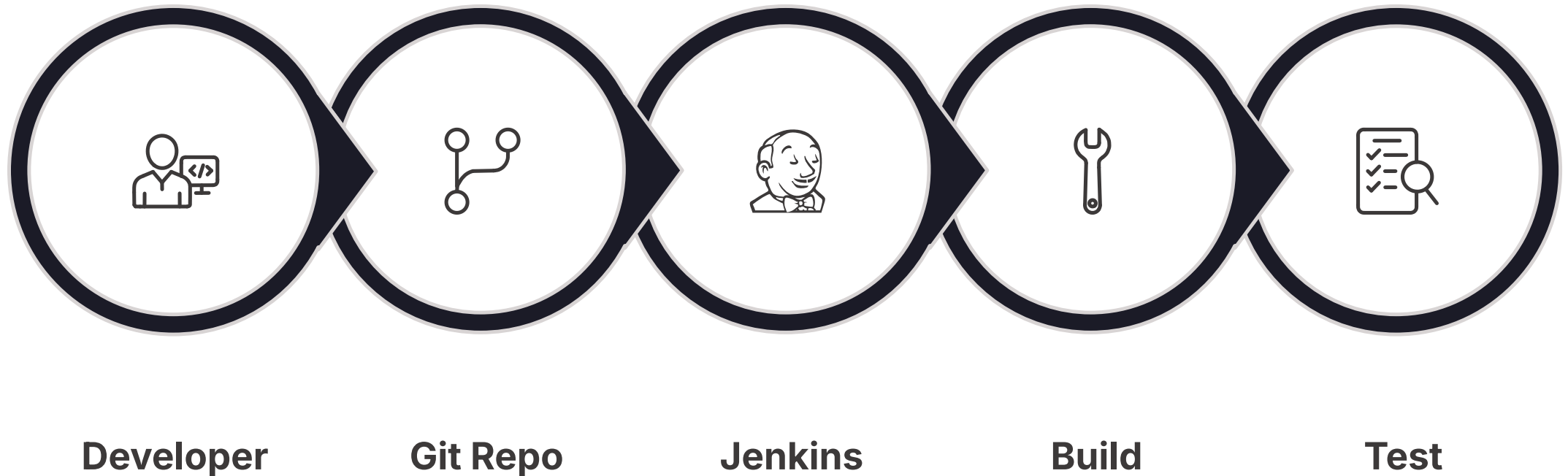
Test

Package

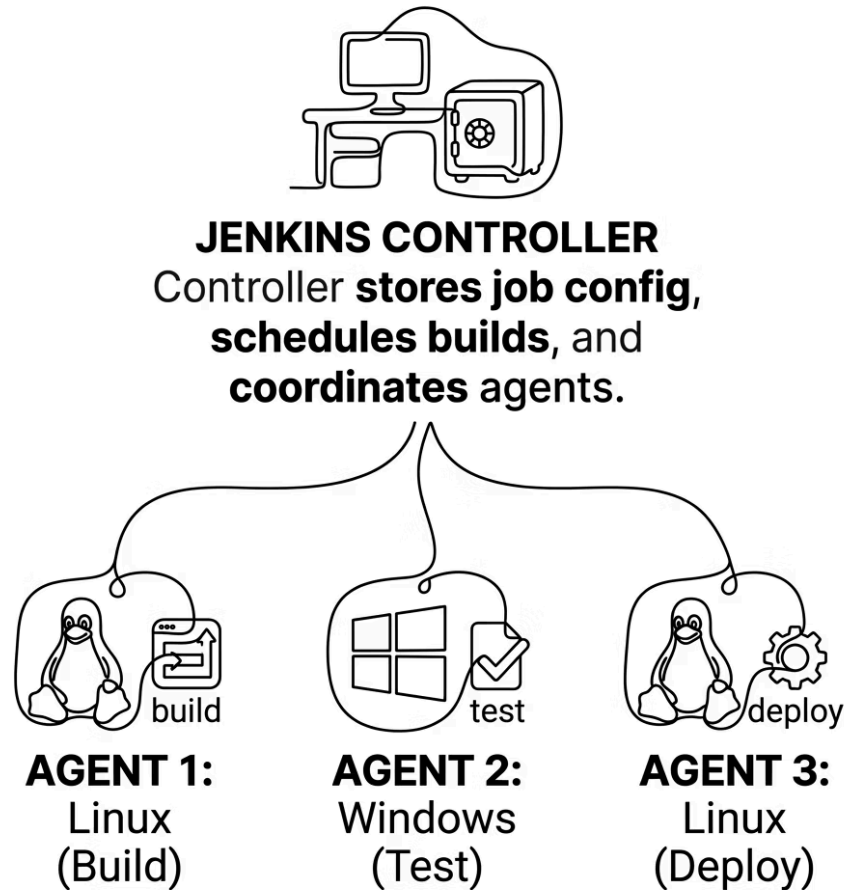
Deploy

Jenkins in the Delivery Lifecycle

Jenkins connects development and DevOps tools into a single automated workflow.



Jenkins Architecture



Jenkins Controller

- Manages Jenkins & stores job configuration
- Schedules builds & coordinates agents
- Maintains build history & information

Jenkins Agent

Executes work assigned by the Controller. Agents can run on Linux, Windows, or Docker environments.

Why Jenkins Agents?

Agents allow Jenkins workloads to execute across different environments simultaneously.



Distributed Workloads

Spread builds across multiple machines



Parallel Execution

Run multiple builds simultaneously



Platform-Specific Builds

Linux, Windows, and Docker agents



Better Scalability

Add agents as workloads grow



Jenkins Core Concepts

Job & Build

Job

The **definition** of work Jenkins should perform.

Build

A **single execution** of a Job — each run gets a unique build number and status.

Simple Analogy

Job = Recipe

The instructions, defined once

Build = Cooking It

One execution of the recipe

Example: **Build-Application** — #101  · #102  · #103  · #104 

Nodes & Executors

Node

A machine or environment where Jenkins can execute work.

Executor

An execution **slot** available on a Node. Multiple executors allow parallel builds on one machine.

Build Queue

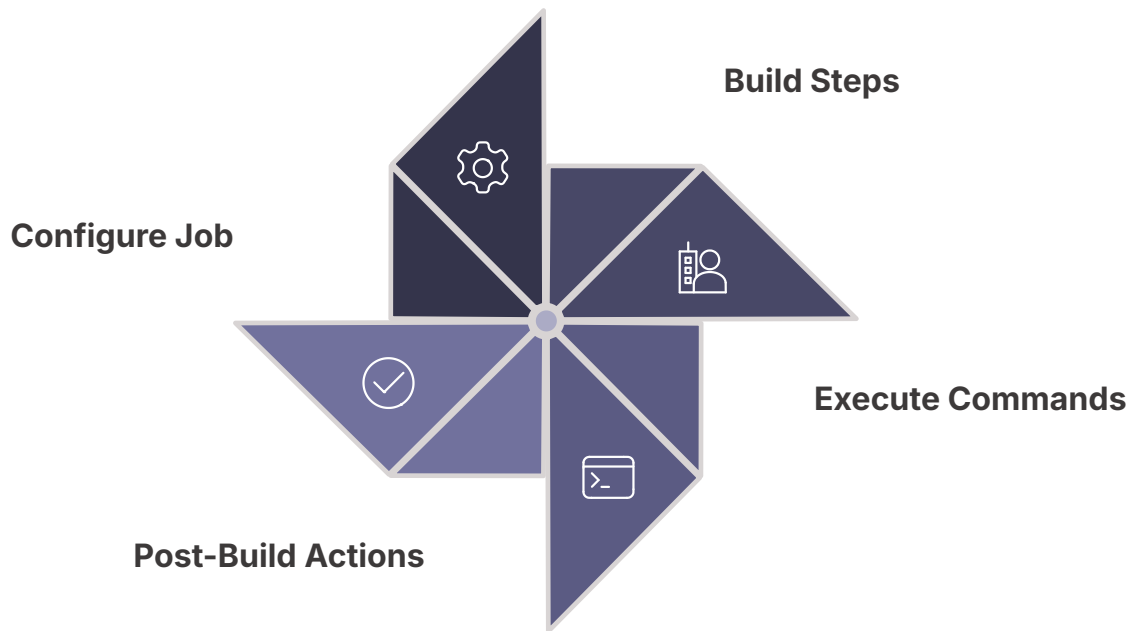
If all suitable executors are busy, new builds **wait in the queue**.



Freestyle Job

What is a Freestyle Project?

A simple Jenkins job configured through the **Jenkins UI** — no code required.



Best Suited For

Simple Automation

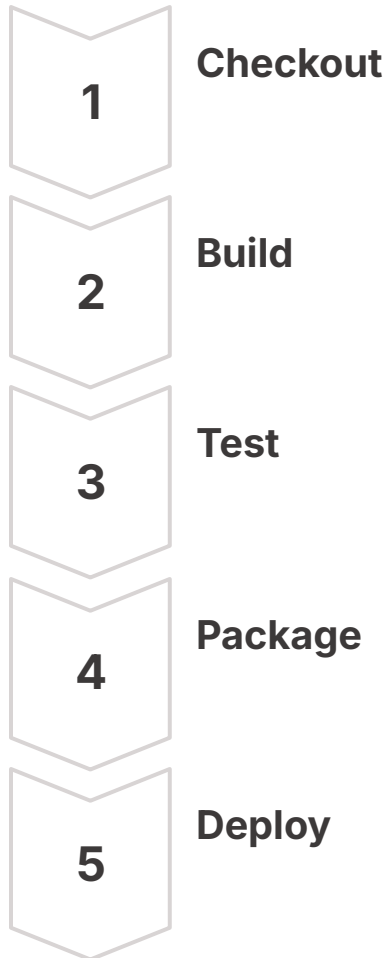
Basic Build Tasks

Learning Jenkins

⚠ Limitation: Complex CI/CD workflows become difficult to maintain through UI configuration alone.

Jenkins Pipeline

A Jenkins Pipeline represents a **complete CI/CD workflow** — defined as code, version-controlled, and repeatable.



Pipeline-as-Code

Store Delivery as Code

A Jenkins Pipeline is defined in a **Jenkinsfile** stored alongside application source code.

```
App Repository/  
|—— src/  
|—— tests/  
|—— pom.xml  
|—— Jenkinsfile
```



Version Controlled

Track changes to your pipeline over time



Code Reviewable

Peer-review pipeline changes like any code



Auditable & Repeatable

Full history, consistent execution every time

Declarative vs. Scripted Pipeline

DECLARATIVE

Structured & Readable

Designed for easier creation and maintenance. **Training focus.**

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        echo 'Building...'  
      }  
    }  
  }  
}
```

SCRIPTED

Flexible & Groovy-Based

More powerful but requires deeper Groovy knowledge.

```
node {  
  stage('Build') {  
    echo 'Building...'  
  }  
}
```

Basic Pipeline Structure

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        echo 'Building application'  
      }  
    }  
    stage('Test') {  
      steps {  
        echo 'Running tests'  
      }  
    }  
  }  
}
```

Four Key Elements

pipeline

Defines the entire Pipeline block

agent

Defines where the Pipeline executes

stages

Groups major workflow phases

steps

Defines actions performed inside a stage

Day 1 Hands-On Labs

1

Lab 1 — Explore Jenkins

Navigate the Dashboard, Manage Jenkins, Nodes, Build History, and Console Output.

2

Lab 2 — Freestyle Job

Create `first-jenkins-job`, add a build step (echo "Hello from Jenkins"), run it, and review build number, status, and console output.

3

Lab 3 — Basic Pipeline

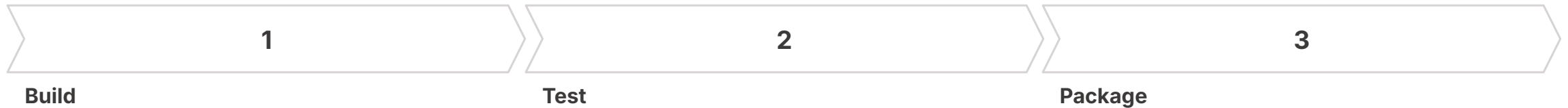
Create a Pipeline job with Build, Test, and Package stages. Review execution and Console Output.



Lab 3 — Pipeline Code

Create a Pipeline job using the following Jenkinsfile and observe the three-stage execution:

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        echo 'Building application...'  
      }  
    }  
    stage('Test') {  
      steps {  
        echo 'Running tests...'  
      }  
    }  
    stage('Package') {  
      steps {  
        echo 'Packaging application...'  
      }  
    }  
  }  
}
```



Day 1 Complete

You now have the foundation to build and automate CI/CD workflows with Jenkins.

CI/CD Concepts

CI, Delivery, Deployment — defined and compared

Jenkins Architecture

Controller, Agents, Nodes, Executors

Jobs & Pipelines

Freestyle vs. Pipeline-as-Code

Hands-On

Built and ran your first Jenkins Pipeline

